field_validator

`@field_validator("...")` is a Pydantic decorator that add custom validation logic. to individual fields.

It always return a value (the original or a transformed one), and raise ValueError to signal invalid input

Basic Syntax

```
from pydantic import BaseModel, field_validator

class User(BaseModel):
    name: str
    age: int

    @field_validator('age')
    @classmethod
    def age_must_be_positive(cls, v):
        if v < 0:
            raise ValueError('age must be positive')
        return v
```

Must include rules:

- It must be a @classmethod
- Must return the (possibly modified) value
- Raise `ValueError` or `AssertionError` to trigger a validation error

Validating Multiple Fields at once

```
@field_validator('name', 'username')
@classmethod
def no_spaces(cls, v):
    if ' ' in v:
        raise ValueError('must not contain spaces')
    return v.strip()
```

`mode` Parameter: Before and After

```
@field_validator('age', mode='before')   # runs before Pydantic's type coercion
@classmethod
def parse_age(cls, v):
    if isinstance(v, str):
        return int(v.replace('years', '').strip())
```

```
    return v

@field_validator('age', mode='after')    # default — runs after type coercion
@classmethod
def check_adult(cls, v):
    if v < 18:
        raise ValueError('must be 18+')
    return v
```

Accessing Other Field Values with `model_validator`

```
from pydantic import model_validator

class Booking(BaseModel):
    check_in: date
    check_out: date

    @model_validator(mode='after')
    def check_dates(self):
        if self.check_out <= self.check_in:
            raise ValueError('check_out must be after check_in')
        return self
```

`each_item` to validate list elements

```
class Cart(BaseModel):
    prices: list[float]

    @field_validator('prices', mode='before', each_item=True)  # Pydantic v1 style
    @classmethod
    def positive_price(cls, v):
        if v < 0:
            raise ValueError('price must be positive')
        return v
```

Returning a Modified Value (Coercion)

Validators can transform the value, not just check in

```
class Profile(BaseModel):
    username: str
    email: str

    @field_validator('username')
    @classmethod
    def lowercase_username(cls, v):
        return v.lower().strip()   # normalizes the value
```

```
    @field_validator('email')
    @classmethod
    def validate_email(cls, v):
        if '@' not in v:
            raise ValueError('invalid email')
        return v.lower()
```